

Workshop

TDD, ADR & AI-Assisted Development

Datum: 25 maart 2026

Auteur: Bas de Kort

Deel 1: Het Probleem; Context Rot

AI heeft geheugenverlies

AI coding assistants werken binnen een **context window**; een beperkt geheugen met een vaste maximale grootte. Elke nieuwe chat begint blanco.

Maar het probleem gaat verder dan alleen sessiewisselingen. Ook **binnen een lopend gesprek** degradeert de context:

Context compressie: Wanneer een gesprek het window nadert, moet de AI oudere berichten samenvatten of laten vallen. Nuances, constraints en eerdere afspraken verdwijnen stilletjes.

Attention decay: Hoe meer tekst er in het window zit, hoe minder gewicht de AI geeft aan informatie die "ver weg" staat. Instructies van 20 berichten geleden worden effectief vergeten, ook al zijn ze technisch nog aanwezig.

Hallucinaties: Wanneer de relevante context verdwenen of verdund is, vult de AI de gaten met plausibel klinkende maar feitelijk onjuiste informatie. De AI *weet niet dat hij het niet meer weet*; hij genereert zelfverzekerd antwoorden op basis van incomplete context.

Het resultaat is **context rot**: een geleidelijk verval van de kwaliteit van AI-output naarmate de sessie langer duurt of tussen sessies door.

In de praktijk betekent dit:

Architectuurbeslissingen gaan verloren (tussen sessies) of raken verdund (binnen een sessie)

De AI maakt dezelfde fouten opnieuw

Code wordt inconsistent omdat patronen vergeten worden

"Waarom is dit zo?" is niet meer te beantwoorden; niet door jou, niet door de AI

Na een lange sessie kan de AI zelfverzekerd code voorstellen die indruist tegen afspraken van eerder in

datzelfde gesprek

Stel je voor dat je een collega hebt die scherp begint, maar gedurende de dag steeds meer vergeet; en het ergste is dat hij niet doorheeft dat hij dingen vergeet.

De oplossing: externaliseer context

De AI vergeet, maar bestanden in je repository niet. De sleutel is context externaliseren in vier lagen:

Laag	Beantwoordt	Wat het vastlegt
Product Requirements Documents (PRDs)	*Wat* en *waarom*	Feature-eisen, doelen en acceptatiecriteria
Plans	*Hoe*	Technische scope, fases en voortgang
Test-Driven Development (TDD)	*Werkt het*	Gedrag, contracten en regressiebeveiliging
Architecture Decision Records (ADRs)	*Waarom zo*	Architectuurbeslissingen en hun onderbouwing

Deel 2: De Vier Lagen in de Praktijk

Laag 1: PRDs; de opdracht

Alles begint met de vraag: **wat** gaan we bouwen en **waarom**? Een PRD formaliseert dat:

Doel; welk probleem lost dit op?

Scope; wat is in en out of scope?

Acceptatiecriteria; wanneer is het klaar?

Zonder PRD begint de AI (en soms de developer) te bouwen op basis van aannames. De PRD is het contract tussen het idee en de implementatie; het voorkomt dat er een technisch plan wordt geschreven voor iets dat nog niet goed gedefinieerd is.

Les: Een PRD hoeft niet zwaar te zijn. Zelfs een half A4 met doel, scope en 5 acceptatiecriteria bespaart uren aan verkeerde implementatie.

Laag 2: Het Plan; de vertaling

Met een PRD in de hand is de volgende stap: **hoe** gaan we dit bouwen? Het plan vertaalt PRD-criteria naar technische specs:

Deelt werk op in fases die in een sessie passen

Maakt elke spec testbaar (directe input voor TDD)

Houdt voortgang bij en legt vast wat **out of scope** is

Het plan is ook een levend document. Tijdens de implementatie van recurring tasks ontdekte TDD een event dispatch gap die niet in het oorspronkelijke plan stond. Het plan werd bijgewerkt met een nieuwe Phase 0, en een ADR geschreven (ADR-013).

Laag 3: TDD; het veiligheidsnet

De specs uit het plan worden tests. TDD is geen "nice to have" bij AI-assisted development; het is het **veiligheidsnet dat voorkomt dat de AI je codebase sloopt**.

Zonder TDD merk je niet dat de AI in sessie 5 iets heeft gebroken dat in sessie 2 werkte. Met TDD worden regressies direct gevangen, en de test suite vormt een levende specificatie die elke sessie overleeft.

Red-Green-Refactor:

RED; schrijf een falende test die het gewenste gedrag beschrijft

GREEN; schrijf de minimale code om de test te laten slagen

REFACTOR; verbeter de code met het vangnet van groene tests

Case study: HasFollowUp trait (ADR-001)

De `HasFollowUp` trait definieerde scopes die direct kolommen `follow_up_date` en `status` bevroegen. Maar die kolommen bestaan alleen op de `follow_ups` tabel; niet op `tasks` of `team_members` die de trait gebruiken. Runtime SQL errors gegarandeerd.

TDD ving dit: de test faalde niet omdat de implementatie fout was, maar omdat het hele **design** fout was. Scopes werden herschreven naar `whereHas`-based queries. De bug was gevonden voordat er ook maar een regel productcode was geschreven.

Les: De tests vonden de bug. Niet de AI, niet code review, niet QA. De falende test dwong een redesign af.

Laag 4: ADRs; het geheugen

Tijdens het bouwen worden beslissingen genomen die afwijken van het plan, of die niet-triviale trade-offs bevatten. Een ADR documenteert drie dingen:

Context; wat was het probleem?

Decision; wat is er besloten en waarom?

Consequences; wat verandert er?

Zonder ADRs kan de AI in een volgende sessie een beslissing terugdraaien die je bewust hebt genomen, of een alternatief voorstellen dat je al hebt afgewezen.

Case study: Event Dispatch Gap (ADR-013)

Bij het implementeren van recurring tasks bleek dat het `TaskStatusChanged` event alleen werd gefired vanuit expliciete dispatch calls. Maar drie andere code paths (`AutoSaveController`, `bulkUpdate()`, `move()`) updaten status via `$task->update()` *zonder* het event te dispatchen. De oplossing: een `TaskObserver` die het event automatisch dispatcht bij elke statuswijziging.

Zonder ADR zou een volgende sessie niet weten dat:

Er bewust een Observer is gekozen boven handmatige dispatch calls (en waarom)

Handmatige dispatches bewust zijn verwijderd

Dit ook een pre-bestaand probleem met een andere listener oploste

De ADR legt niet alleen vast *wat* er is besloten, maar ook welke alternatieven zijn afgewezen en waarom. Dat voorkomt dat de AI (of een developer) dezelfde discussie opnieuw voert.

Vuistregel: Als iemand later zou kunnen vragen "waarom is dit zo?"; schrijf een ADR.

De feedback loop

Probleem	Oplossing
AI vergeet vorige sessie	ADRs + plan overleven sessies
AI breekt bestaande code	Tests vangen regressies direct
AI gebruikt andere patronen	Plan + codebase scan dwingen patronen af
AI voegt ongeplande features toe	Plan met "Out of Scope" sectie
AI draait bewuste keuzes terug	ADRs documenteren het *waarom*

Deel 3: Toepassen en Valkuilen

De vijf belangrijkste lessen

Als het niet in een bestand staat, bestaat het niet. Elke AI-sessie begint leeg. Context die alleen in je hoofd zit, gaat verloren.

Maak specs testbaar. Als je het niet in een test kunt uitdrukken, is de spec te vaag. Dit is waar plan en TDD samenkomen.

Schrijf ADRs op het moment van de beslissing. Achteraf mis je de alternatieven die je hebt overwogen.

Houd sessies klein. Context windows hebben limieten. Kleine fases in het plan houden sessies beheersbaar en voorkomen context compressie.

Jij blijft de architect. De AI is een krachtig hulpmiddel, maar review, denk mee, en stel vragen. Blind vertrouwen leidt tot blinde vlekken.

Hoe begin je?

Je hebt geen speciale tooling nodig om dit toe te passen:

Start elke AI-sessie met context laden; geef de AI bij de start de relevante ADRs, het plan, en laat hem de tests lezen.

Gebruik een `CLAUDE.md` (of equivalent); een bestand in je repo dat de AI automatisch leest bij elke sessie. Hierin staan conventies, actieve plannen, en verwijzingen naar ADRs.

Houd ADRs klein; een ADR per beslissing, niet een mega-document per feature.

Gebruik het plan als communicatiemiddel; niet alleen voor de AI, maar ook voor je team.

Mithril in cijfers

Metric	Waarde
Commits	418
Testbestanden	187
Testfuncties	1091
ADRs	28
Implementatieplannen	19
Tijdspanne	3 weken

Gebouwd met consistente architectuur ondanks tientallen sessiewisselingen.

Deel 4: ANVIL; de vier lagen geautomatiseerd

Van handmatig naar georchestreed

De vier lagen uit deel 2 werken, maar vereisen discipline: je moet zelf onthouden een PRD te schrijven, een plan te maken, TDD te volgen, en ADRs bij te houden. In de praktijk slijpt dat; vooral onder tijdsdruk.

ANVIL (Automated Nexus for Verified Iterative Lifecycles) is een skill voor Claude Code die deze flow probeert te automatiseren. Het is in actieve ontwikkeling en door mij gebouwd vanuit de lessen van het Mithril project.

Hoe het werkt

ANVIL is een orchestrator die de volledige lifecycle aanstuurt via gespecialiseerde rollen:

Elke rol is een AI-agent met een specifieke opdracht en regels. De orchestrator stuurt ze aan, bewaakt de voortgang, en zorgt dat de gebruiker altijd de controle houdt.

Zes rollen

Rol	Verantwoordelijkheid
Analyst	Haalt requirements op uit externe bronnen (bijv. Jira)
Specifier	Schrijft en reviewt PRDs
Architect	Maakt implementatieplannen met fases en specs
Builder	Bouwt via TDD (red-green-refactor), kan parallel werken in agent teams
Auditor	OWASP security scans op de codebase
Scribe	Genereert documenten vanuit de opgeleverde resultaten

Automatische bewaking

Naast de rollen heeft ANVIL twee Claude Code hooks (PostToolUse) die real-time afdwingen wat anders op discipline leunt:

TDD Enforce; waarschuwt direct wanneer je een bronbestand wijzigt zonder dat er testwijzigingen in de working tree staan. Je krijgt de melding op het moment van de edit, niet pas bij een commit. Dit dwingt de Red-Green volgorde af: eerst tests, dan implementatie.

ADR Watch; waarschuwt wanneer een bestand wordt gewijzigd dat buiten het actieve plan valt. Voorkomt onbewuste scope creep.

Beide hooks zijn informatief (ze blokkeren niet) en worden automatisch geïnstalleerd via ``/anvil init`` in ``.claude/settings.local.json``.

Hoe ANVIL context rot minimaliseert

De vier lagen uit deel 2 bestrijden context rot op documentniveau. ANVIL voegt daar drie mechanismen aan toe die specifiek op het *verval binnen en tussen sessies* gericht zijn:

1. State op disk, niet in het geheugen

ANVIL leidt de lifecycle state af uit bestanden: de status in een plan, de aanwezigheid van PRDs, de voortgang van fases. Niets hangt af van wat de AI "onthoudt". Wanneer een sessie eindigt (of het context window vol raakt), schrijft ANVIL een session handoff met de huidige taak, voltooide werk, en volgende stappen. De volgende sessie leest dat bestand en pikt het werk op alsof er geen onderbreking was.

2. Kleine, gefocuste agents

In plaats van een enkele AI-sessie die steeds langer wordt (en steeds meer vergeet), splitst ANVIL werk op in gespecialiseerde agents. Een Builder-agent krijgt alleen de specs voor zijn fase, de relevante ADRs, en de projectconventies. Zijn context window bevat precies wat hij nodig heeft; niet de volledige geschiedenis van het project. Dit voorkomt attention decay doordat elke agent een kort, gefocust window heeft. Een bijkomend voordeel: kleinere context windows betekenen minder tokens per agent-aanroep, wat de API-kosten drukt en de responstijd verkort. Agents werken sneller omdat ze minder tekst hoeven te verwerken en minder last hebben van ruis.

Een kanttekening: de orchestrator zelf accumuleert wel state gedurende de sessie; elke agent-dispatch, elk resultaat en elke gebruikersbeslissing vergroot zijn context. Bij complexe plannen met veel fases kan de orchestrator op termijn tegen dezelfde context-limieten aanlopen die de agents juist vermijden. Context management blijft dus wel belangrijk om te monitoren.

3. Mechanische bewaking vervangt geheugen

De twee hooks (TDD Enforce en ADR Watch) vangen fouten die ontstaan door context rot *op het moment dat ze gebeuren*. Als de AI halverwege een sessie vergeet dat er een actief plan is en buiten scope begint te werken, waarschuwt ADR Watch direct. Als de AI vergeet dat TDD verplicht is en implementatiecode schrijft zonder tests, grijpt TDD Enforce in. Dit zijn geen herinneringen die de AI kan vergeten; het zijn mechanische checks die onafhankelijk van de AI draaien.

Context rot probleem	ANVIL oplossing
Sessiewissel verliest voortgang	Session handoff + state op disk
Lang window; attention decay	Kleine agents met gefocuste context
AI vergeet eigen regels	Hooks die onafhankelijk van de AI draaien
AI herhaalt afgewezen keuzes	ADRs geladen in elke agent prompt
Scope creep door verdunde context	ADR Watch detecteert out-of-plan edits

De kern blijft

ANVIL automatiseert de *flow*, maar de principes zijn dezelfde als in deel 2: PRDs, plannen, tests en beslissingen; vastgelegd in bestanden die elke sessie overleven. De tooling maakt het makkelijker om de discipline vol te houden, maar je kunt dezelfde aanpak toepassen met elk AI-hulpmiddel en een teksteditor.

Zelf uitproberen

ANVIL is op dit moment nog in actieve ontwikkeling, maar ik zal de skill binnenkort beschikbaar stellen voor iedereen binnen Proud Nerds die hem wil uitproberen. Houd er rekening mee dat het een Claude Code skill is; je hebt een werkende Claude Code setup nodig om ermee aan de slag te gaan.

Discussie / Q&A (resterende tijd)

Hoe past dit in jullie huidige workflow?

Welke onderdelen zijn direct toepasbaar; ook zonder AI?

Wat is de overhead vs. de tijdsbesparing?